Technical PRD: Tempest Trends

1. Executive Summary

Tempest Trends is a responsive, high-performance web-based weather application. The core vision is to provide users with accurate current, hourly, and 7-day weather forecasts through a completely unique, highly polished UI. For the MVP, it emphasizes speed and reduced user friction by utilizing browser `localStorage` to manage favorite cities without requiring account creation. The application bridges robust data (via OpenWeatherMap) with modern front-end design to deliver immediate value to the user.

2. User Personas & Stories

Persona 1: The Daily Commuter (Alex)

Needs quick, actionable weather data for the immediate hours ahead.

Role	Action	Benefit
Commuter	Open the app and auto-detect my location	I can instantly see if I need an umbrella right now.
Commuter	View the 12-hour hourly forecast	I can decide what to wear for the commute home.
Commuter	Save my work city to "Favorites"	I can check the weather at the office without typing the city name again.

Persona 2: The Outdoor Planner (Sam)

Needs detailed, multi-day forecasting to plan weekend activities.

Role	Action	Benefit
Planner	Search for a remote zip code or city	I can check the exact forecast for my hiking destination.
Planner	View the 7-day extended forecast	I can decide which day has the most favorable conditions for an outing.
Planner	Read specific data like wind speed and humidity	I can properly pack and prepare for the exact environmental conditions.

3. Functional Requirements & Acceptance Criteria

1. **Geolocation & Auto-Load**
    - **Technical Acceptance Criteria:** On first load, the browser must prompt for Geolocation API access. If granted, the app fetches coordinates and passes them to the weather API. If denied, the app gracefully degrades, displaying a default search state.
2. **City Search & Autocomplete**
    - **Technical Acceptance Criteria:** The search bar must accept City Name or Zip Code. It must implement a 300ms debounce before querying the Geocoding API to prevent rate-limit exhaustion.
3. **Weather Dashboard Display**
    - **Technical Acceptance Criteria:** The UI must render current temperature, "feels like" temperature, humidity, wind speed, and an icon representing current conditions. Data must parse correctly from the OpenWeatherMap JSON response.
4. **Hourly & 7-Day Forecasts**
    - **Technical Acceptance Criteria:** The application must map the 5-day/3-hour or One Call API response into an easily readable scrolling horizontal list (hourly) and vertical stack (daily).
5. **Local State Management (Favorites)**
    - **Technical Acceptance Criteria:** Users can click a "Star" icon to save a location. The app must serialize an array of `Location` objects and store it in `window.localStorage`. On subsequent loads, the app must hydrate the state from `localStorage` to populate the favorites drawer.

4. Implementation Roadmap

- **Phase 1: MVP Core (Current Focus)**
    1. Initialize Next.js project and configure Tailwind CSS/Shadcn UI.
    2. Set up Next.js API routes (Backend-for-Frontend) to securely fetch OpenWeatherMap data.
    3. Build core UI components (Search, Current Weather Card, Forecast Lists).
    4. Implement `localStorage` logic and Geolocation API.
    5. Deploy to Vercel for live testing.
- **Phase 2: Scaling (Outlook)**

1. Migrate `localStorage` to a hosted database (e.g., PostgreSQL via Supabase) with OAuth2 authentication.
2. Implement Interactive Radar Maps.
3. Configure severe weather Push Notifications (Service Workers/PWA).

5. Technical Architecture & UI/UX

- **Stack:** * **Front-End:** Next.js (React), TypeScript.
  - **Back-End (BFF):** Next.js API Routes (Node.js edge functions).
- **UI/UX Framework:** Tailwind CSS combined with **Shadcn/ui** for highly customizable, accessible, and unstyled base components to ensure a unique look.
- **API Standard:** All internal Next.js API routes handling the weather data proxy must be documented using the **OpenAPI 3.0/Swagger** specification for contract-first development.
- **Security:** * The OpenWeatherMap API key must strictly reside in server-side environment variables (`.env.local`) and never be exposed to the client bundle.
  - Basic rate-limiting applied to Next.js API routes to prevent abuse.
- **Infrastructure/DevOps:** * **Hosting & CI/CD:** Vercel (provides zero-config CI/CD from GitHub, edge caching, and cost-effective scaling).
  - **Containerization:** A `Dockerfile` will be included in the repository for local development parity and platform-agnostic portability, though Vercel serverless is the primary deployment target.
- **External System Dependencies:**
  - **OpenWeatherMap API:** Sole provider for Geocoding (text-to-coordinates), Current Weather, and Forecast data.

6. Data Schema

Because the MVP relies on local storage and API responses rather than a relational database, this ER diagram models the JSON data structures and client-side relationships.

```
erDiagram
    LOCAL-STORAGE ||--o{ FAVORITE-CITY : stores
    FAVORITE-CITY {
        string id
        string cityName
        string countryCode
        float lat
        float lon
    }
```

```
    WEATHER-RESPONSE ||--o{ HOURLY-FORECAST : contains
    WEATHER-RESPONSE ||--o{ DAILY-FORECAST : contains
    WEATHER-RESPONSE {
        float currentTemp
        float feelsLike
        int humidity
        float windSpeed
        string conditions
        string iconCode
    }
    HOURLY-FORECAST {
        string timestamp
        float temp
        string iconCode
    }
    DAILY-FORECAST {
        string date
        float maxTemp
        float minTemp
        string conditions
    }
```

7. User Flow & State Logic

```
flowchart TD
    A[User visits Tempest Trends] --> B{Local Storage has Favorites?}
    B -- Yes --> C[Load default favorite city]
    B -- No --> D{Geolocation Granted?}
    D -- Yes --> E[Get User Coordinates]
    D -- No --> F[Display Search Bar]
    F --> G[User inputs City Name]
    G --> H[Fetch Geocoding Data]
    H --> I[Fetch Weather via Next.js API]
    E --> I
    C --> I
    I --> J{API Response 200?}
    J -- Yes --> K[Render Weather Dashboard]
    J -- No --> L[Show Error UI / Retry logic]
    K --> M[User toggles 'Save to Favorites']
    M --> N[Update window.localStorage]
```

8. Success Metrics

- **Latency:** API route response time (OpenWeather proxy) < 600ms.
- **Client Performance:** Core Web Vitals targets: Largest Contentful Paint (LCP) < 2.0s, Cumulative Layout Shift (CLS) < 0.1.
- **Error Rate:** Geolocation failures and API timeouts < 2% of total sessions.

- **User Engagement:** > 30% of users interact with the "Save to Favorites" feature within their first 3 sessions.